

GCN-LRP explanation

exploring latent attention of graph convolutional networks

Fundamental Team

김채현, 양현모, 이근배, 이재윤

C O N T E N T S

1 Introduction

2 Background

3 Method

- 2.1 Graph Convolutional Networks
- 2.2 Layer-wise Relevance Propagation

4 Performance Evaluation

- 4.1 Setting
- 4.2 Results

5 Conclusion

Motivation

1. Graph Convolutional Networks (GCNs)

: successfully applied to many graph data on various learning tasks (ex. node classification)

BUT limit understanding of the **internal logic** and decision **patterns** of GCNs

2. Explainable Artificial Intelligence (XAI)

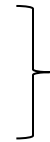
: explain how the input relevant to the decisions of deep neural networks

Class Activation Mapping (CAM)

Excitation Backpropagation (EB)

gradient-weighted CAM (Grad-CAM)

Layer-wise Relevance Propagation (LRP)



adapted to explain GCN
("Explainability methods for graph convolutional neural networks")

Motivation

Layer-wise Relevance Propagation (LRP)

: used to figure out how the prediction values generated from original input values which brings the explainability about how input data support the prediction in the trained model

different for Euclidean data (ex. image data) and non-Euclidean data (ex. graph data)

Image data : we can use the output of LRP to observe and confirm pixel contributions
(images are grid-structured data in Euclidean space, the explaining outputs of LRP are only relevant to the grid value and learning weights)

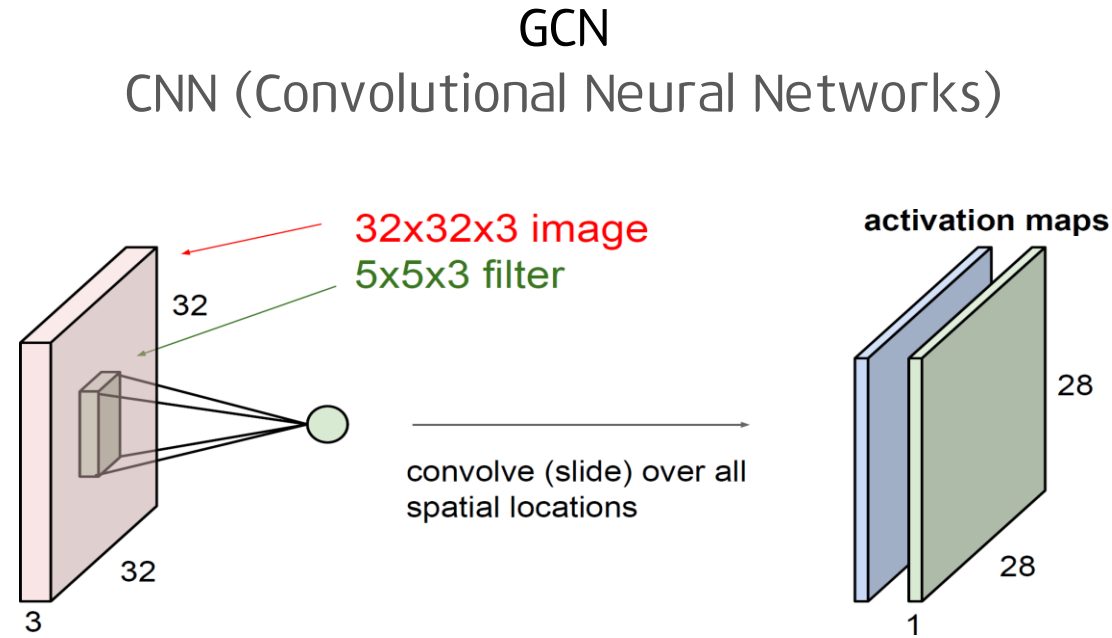
Graph data : graph structure will also influence the output value of the explaining methods

$\therefore$ can not use original LRP method to explain GCN directly

Proposed Method

Contribution

1. propose a new LRP based explanation method ([GCN-LRP](#)) for GCNs
: explain the GCN model on feature level with the contributions of each feature from nodes on local group or category group; and explain the GCN model on node level with the contributions of each node
2. experimentally identify [latent attentions](#) of GCNs
 - (i) GCNs pay more attention to the classified node when comparing with its neighbors
 - (ii) GCNs do not pay equal attention to all the neighboring nodes
3. theoretical analysis
 - (i) the latent attentions come from the recursively aggregating of GCNs
 - (ii) the neighboring nodes, which share enough neighbors with classified node, would receive more attention than other neighbors
 - (iii) the latent attention could hardly be changed by model training



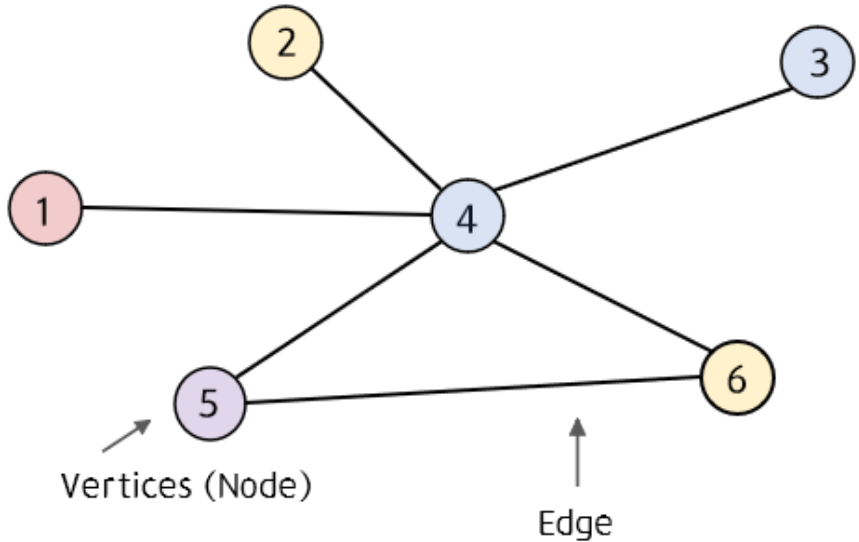
After passing through CNN, one layer changes the values in the activation map

- After receiving an activation map and passing through the convolution layer, values in the activation map is updated
- values in the activation map determine the status of the image or activation map

Role of [convolution layer](#)

: The value of the activation map is information that contains the characteristics of the image, and the value of the activation map is updated when passing the layer

GCN
Graph Data



Vertices (Node)
Node Feature matrix **X**

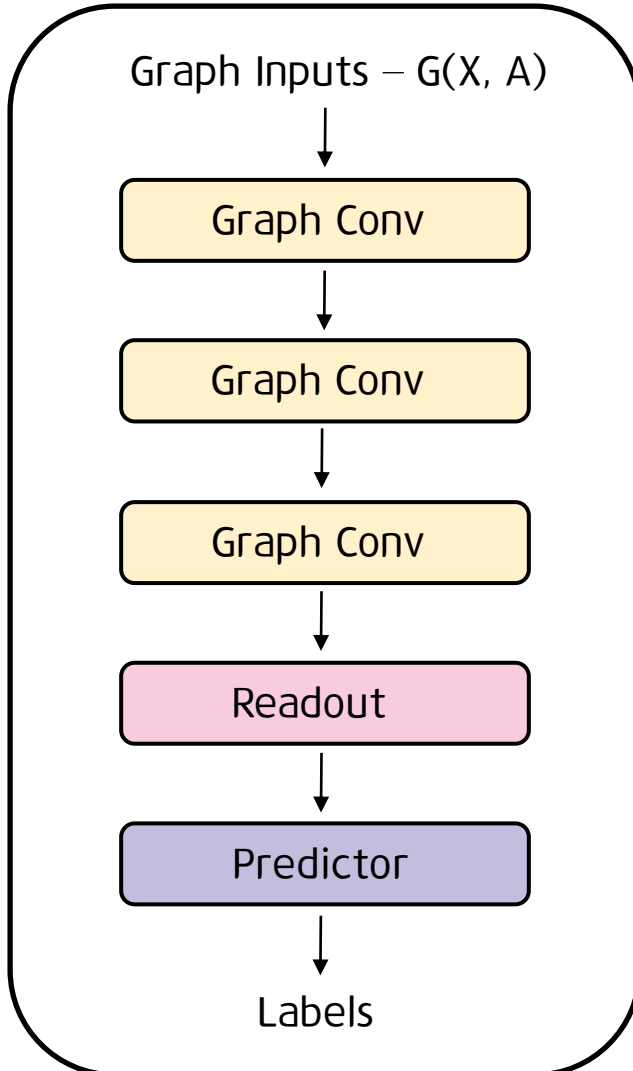
	number of features				
node 1	1	1	1	0	0
node 2	1	1	1	1	1
node 3	1	0	1	1	1
node 4	1	0	1	1	1
node 5	0	1	1	0	1
node 6	1	1	1	1	1

Edge
Adjacency matrix **A**

	n1	n2	n3	n4	n5	n6
node 1	0	0	0	1	0	0
node 2	0	0	0	1	0	0
node 3	0	0	0	1	0	0
node 4	1	1	1	0	1	1
node 5	0	0	0	1	0	1
node 6	0	0	0	1	1	0

GCN

Graph Convolutional Networks



Inspired by convolutional neural network(CNN) successfully used in computer vision

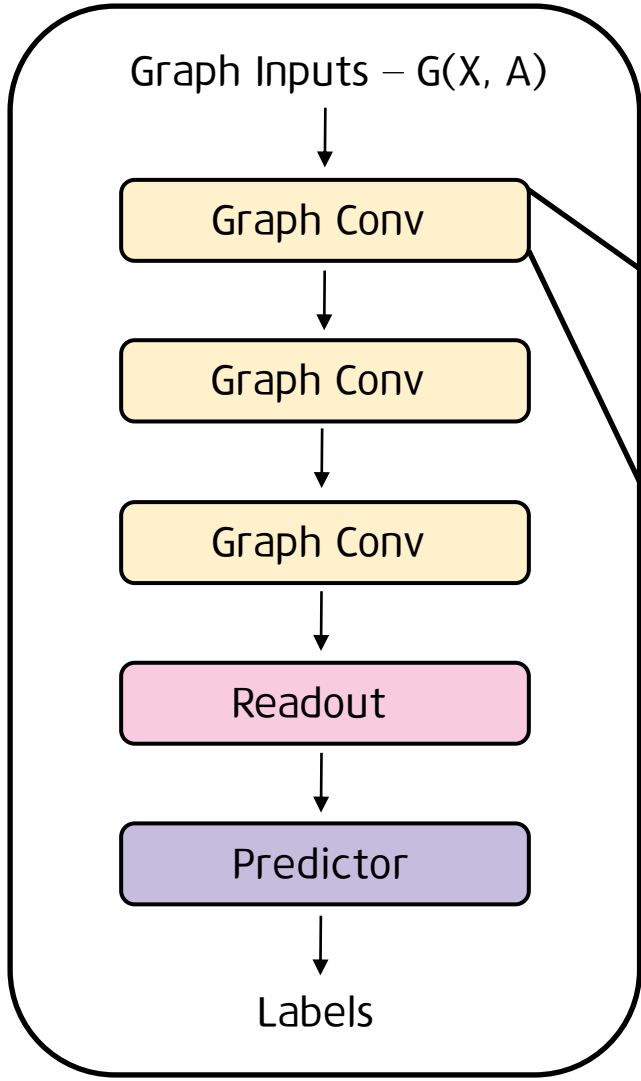
↓
extend **convolutional operations** onto graph as an aggregation method

How to determine the information in the graph

: The value contained in each node (information contained in each node feature matrix) determines node information without changing the shape of the graph

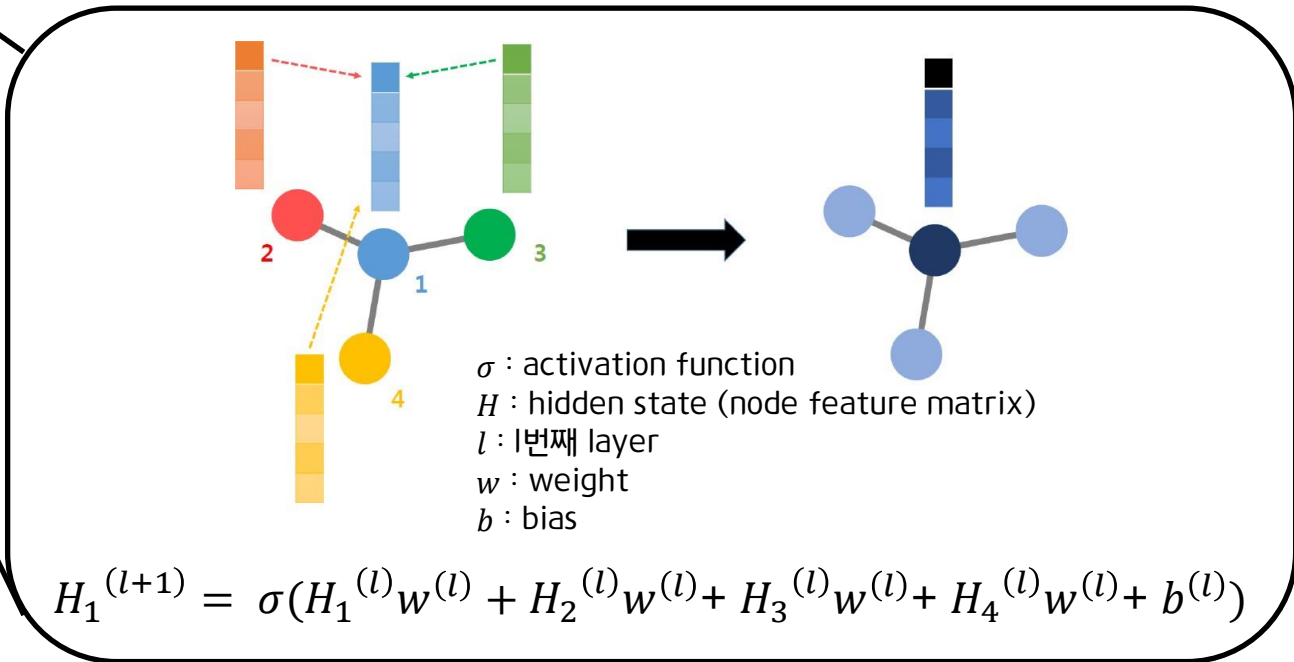
∴ Change the values in the **node feature matrix** to be updated

GCN
Graph Convolutional Networks

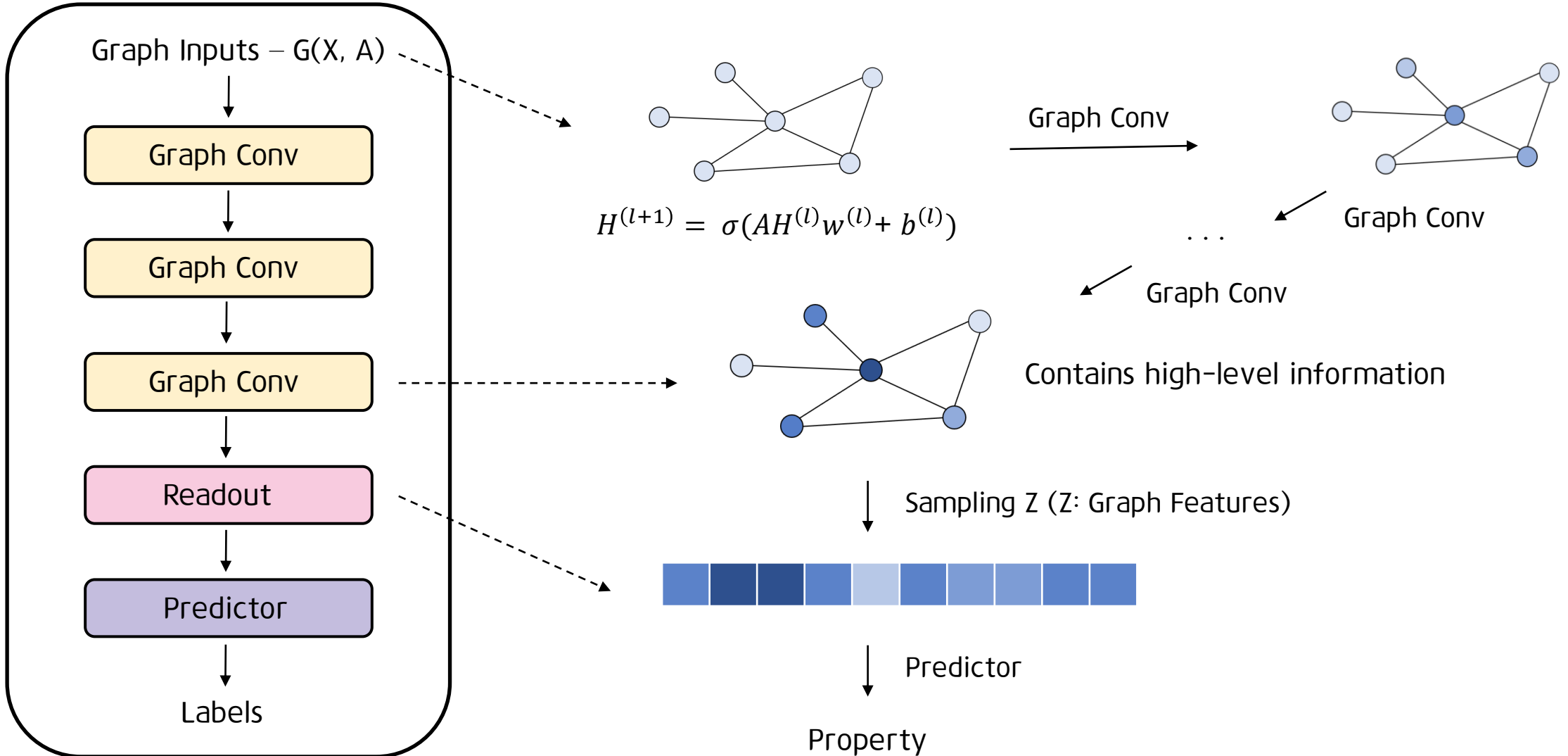


Convolution Layer

- 1. **local features** can be extracted : self node(1) receive only the information of connected nodes(2, 3, 4)
- 2. **weight sharing** : multiplied by the same weight to extract general information (computational cost ↓ , efficiency ↑)



GCN
Graph Convolutional Networks



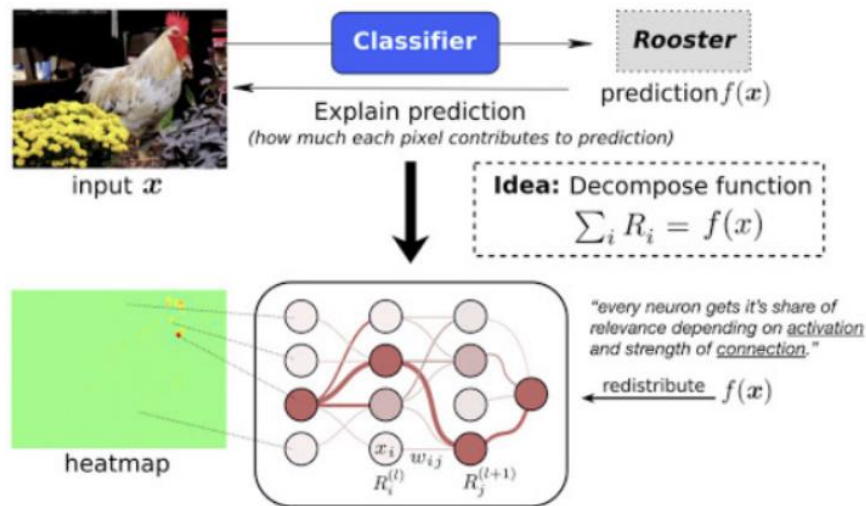
LRP

Layer-wise Relevance Propagation

Explainability in computer vision – Attribution propagation method

Understanding why the Neural Network model made such a decision through [decomposition](#)

If the input x consists of d dimension, it is assumed that each feature of d dimension has different influence in driving the final output, and this relevance score is calculated and analyzed



x : sample image

$f(x)$: prediction of image x

R_i : relevance score of each dimension

degree to which pixel of image x contributed to obtain prediction $f(x)$

LRP result [heatmap](#)

Display the contribution(Relevance Score) of each pixel of image x
By looking at the top right(rooster's beak or head) outputs the prediction for x as “Rooster”

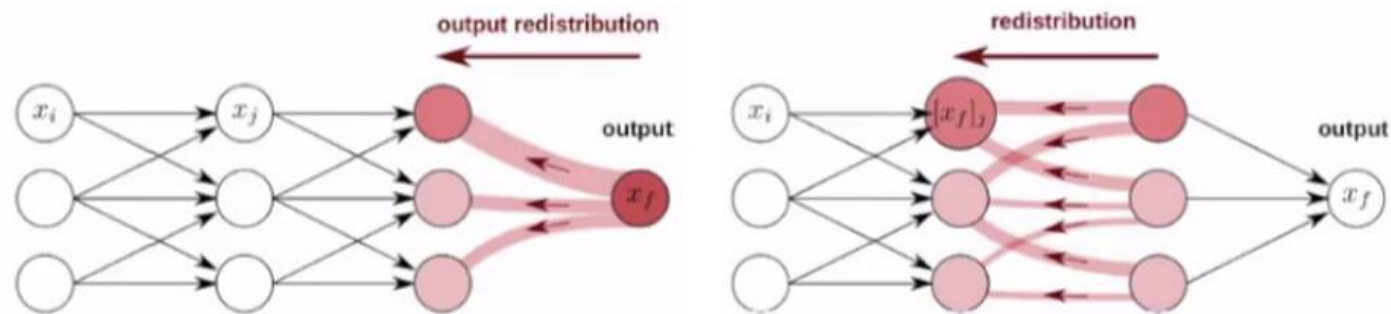
LRP

Layer-wise Relevance Propagation

Redistribute the contribution of Relevance Score from output to input in a top-down manner

Basic assumptions of LRP

- Each neuron has a certain relevance
- Contribution is redistributed from the output of each neural to the input in a top-down manner
- Preservation of contribution when distributed



LRP

Layer-wise Relevance Propagation

LRP calculate each input neuron's contribution to the activation in neural networks, and these contributions are propagated back layer-wise as relevance scores

Because the layer-wise propagation relies on model structure and trained weights, LRP method explain how trained model treats all related input neurons when it makes decision

For the typical neural network layer without bias :

$$R_j^l = \sum_k \frac{a_j^l w_{jk}^l}{\sum_{0,j} a_j^l w_{jk}^l} R_k^{l+1}, A^l = \{a_j^l | j = 0, \dots, m\}, W^l = \{w_{jk}^l | j = 0, \dots, m; k = 0, \dots, n\} \quad (1)$$

Z^l : input vector of l layer

R_j^l : relevance score of neuron h_j^l

$\sum_k$: sums over the neurons related to neuron h_j^l in layer $l + 1$

$\sum_{0,j}$: sums over the neurons related to neurons $\{h_j^{l+1}\}$ in layer $l + 1$

Q & A

Proposed Method

Explaining GCN with LRP

Algorithm 1: GCN-LRP for node classification tasks of GCNs.

Input: Activations $A = (X, a_1, \dots, a_l)$, weights $W = (w_1, \dots, w_l)$, bias $B = (b_1, \dots, b_l)$, normalized graph Laplacian L , classified node index k

Output: The relevance scores matrix R_0

```

1:  $LA \leftarrow [LX, LA_1, \dots, LA_{l-1}]$ 
2:  $Rs \leftarrow [None] * L + [A_l]$ 
3:  $nodes = [k]$ 
4: for  $i = 1, 2, \dots, l$  do
5:   // calculate relevance scores for every layer
6:    $R \leftarrow zeros(A[-i-1].shape)$ 
7:    $nodes\_next \leftarrow set([])$ 
8:   for  $node$  in  $nodes$  do
9:     // use (5) to calculate preliminary relevance scores.
10:     $z \leftarrow matmul(LA[-i][node, :]) + B[-i]$ 
11:     $s \leftarrow Rs[-i][node, :]/z$ 
12:     $c \leftarrow matmul(s, W[-i].T)$ 
13:     $r \leftarrow LA[-i][node, :] * c$ 
14:
15:    // use (6) to calculate final relevance scores.
16:     $index\_nonzero \leftarrow \text{find all nonzero element's index in } L[node, :]$ 
17:     $nodes\_next \leftarrow nodes\_next + set(index\_nonzero)$ 
18:
19:     $R\_node \leftarrow zeros(A[-i-1].shape)$ 
20:     $L\_vec \leftarrow L[node, :]$ 
21:     $vecs \leftarrow [A[-i-1][j, :] * L\_vec[j] \text{ for } j \text{ in } index\_nonzero]$ 
22:     $sum\_vec \leftarrow sum(vecs, 0)$ 
23:     $ratio\_vecs \leftarrow [vec/sum\_vec \text{ for } vec \text{ in } vecs]$ 
24:    for  $k$  in  $index\_nonzero$  do
25:       $R\_node[index\_nonzero[k]] \leftarrow r * ratio\_vecs[k]$ 
26:    end
27:     $R \leftarrow R + R\_node$ 
28:  end
29:   $Rs[-i-1] \leftarrow R$ 
30:   $nodes \leftarrow nodes\_next$ 
31: end
32:  $R_0 = R[0]$ 

```

Treat the process of layer relevance propagation on GCN as **inverse process** of forward reasoning through GCN

Forward Reasoning

1. each node i calculates the weighted sum of all neighboring nodes' representations
weight for each neighboring node j equals the value locate in the row i and column j of the normalized graph Laplacian L
2. each node transforms its representation by multiplying W

Proposed Method

Explaining GCN with LRP

Algorithm 1: GCN-LRP for node classification tasks of GCNs.

Input: Activations $A = (X, a_1, \dots, a_l)$, weights $W = (w_1, \dots, w_l)$, bias $B = (b_1, \dots, b_l)$, normalized graph Laplacian L , classified node index k

Output: The relevance scores matrix R_0

```

1:  $LA \leftarrow [LX, LA_1, \dots, LA_{l-1}]$ 
2:  $Rs \leftarrow [None] * L + [A_l]$ 
3:  $nodes = [k]$ 
4: for  $i = 1, 2, \dots, l$  do
5:   // calculate relevance scores for every layer
6:    $R \leftarrow zeros(A[-i-1].shape)$ 
7:    $nodes\_next \leftarrow set([])$ 
8:   for  $node$  in  $nodes$  do
9:     // use (5) to calculate preliminary relevance scores.
10:     $z \leftarrow matmul(LA[-i][node, :]) + B[-i]$ 
11:     $s \leftarrow Rs[-i][node, :]/z$ 
12:     $c \leftarrow matmul(s, W[-i].T)$ 
13:     $r \leftarrow LA[-i][node, :] * c$ 
14:
15:   // use (6) to calculate final relevance scores.
16:    $index\_nonzero \leftarrow$  find all nonzero element's index in  $L[node, :]$ 
17:    $nodes\_next \leftarrow node\_next + set(index\_nonzero)$ 
18:
19:    $R\_node \leftarrow zeros(A[-i-1].shape)$ 
20:    $L\_vec \leftarrow L[node, :]$ 
21:    $vecs \leftarrow [A[-i-1][j, :] * L\_vec[j] \text{ for } j \text{ in } index\_nonzero]$ 
22:    $sum\_vec \leftarrow sum(vecs, 0)$ 
23:    $ratio\_vecs \leftarrow [vec/sum\_vec \text{ for } vec \text{ in } vecs]$ 
24:   for  $k$  in  $index\_nonzero$  do
25:      $R\_node[index\_nonzero[k]] \leftarrow r * ratio\_vecs[k]$ 
26:   end
27:    $R \leftarrow R + R\_node$ 
28: end
29:  $Rs[-i-1] \leftarrow R$ 
30:  $nodes \leftarrow nodes\_next$ 
31: end
32:  $R_0 = R[0]$ 

```

Calculate preliminary relevance scores of node features

$$\tilde{R}_j^l = \sum_k \frac{\tilde{x}_j^l w_{jk}^l}{\sum_{0,j} \tilde{x}_j^l w_{jk}^l} R_k^{l+1}, \tilde{X}^l = \{\tilde{x}_j^l | j = 0, \dots, m\}, \tilde{W} = \{\tilde{w}_{jk}^l | j = 0, \dots, m; k = 0, \dots, n\} \quad (5)$$

$$\tilde{X} = \tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} X$$

$\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}}$: normalized adjacency matrix with added self-connections

X : node feature matrix

$\tilde{W}$: trainable weights

propagate relevance scores only in node's feature fields just like the normal LRP method

Proposed Method

Explaining GCN with LRP

Algorithm 1: GCN-LRP for node classification tasks of GCNs.

Input: Activations $A = (X, a_1, \dots, a_l)$, weights $W = (w_1, \dots, w_l)$, bias $B = (b_1, \dots, b_l)$, normalized graph Laplacian L , classified node index k

Output: The relevance scores matrix R_0

```

1:  $LA \leftarrow [LX, LA_1, \dots, LA_{l-1}]$ 
2:  $Rs \leftarrow [None] * L + [A_l]$ 
3:  $nodes = [k]$ 
4: for  $i = 1, 2, \dots, l$  do
5:   // calculate relevance scores for every layer
6:    $R \leftarrow zeros(A[-i-1].shape)$ 
7:    $nodes\_next \leftarrow set([])$ 
8:   for  $node$  in  $nodes$  do
9:     // use (5) to calculate preliminary relevance scores.
10:     $z \leftarrow matmul(LA[-i][node, :]) + B[-i]$ 
11:     $s \leftarrow Rs[-i][node, :]/z$ 
12:     $c \leftarrow matmul(s, W[-i].T)$ 
13:     $r \leftarrow LA[-i][node, :] * c$ 
14:
15:    // use (6) to calculate final relevance scores.
16:     $index\_nonzero \leftarrow$  find all nonzero element's index in  $L[node, :]$ 
17:     $nodes\_next \leftarrow nodes\_next + set(index\_nonzero)$ 
18:
19:     $R\_node \leftarrow zeros(A[-i-1].shape)$ 
20:     $L\_vec \leftarrow L[node, :]$ 
21:     $vecs \leftarrow [A[-i-1][j, :] * L\_vec[j] \text{ for } j \text{ in } index\_nonzero]$ 
22:     $sum\_vec \leftarrow sum(vecs, 0)$ 
23:     $ratio\_vecs \leftarrow [vec/sum\_vec \text{ for } vec \text{ in } vecs]$ 
24:    for  $k$  in  $index\_nonzero$  do
25:       $R\_node[index\_nonzero[k]] \leftarrow r * ratio\_vecs[k]$ 
26:    end
27:     $R \leftarrow R + R\_node$ 
28:  end
29:   $Rs[-i-1] \leftarrow R$ 
30:   $nodes \leftarrow nodes\_next$ 
31: end
32:  $R_0 = R[0]$ 

```

Calculate final relevance scores about this node's all neighboring nodes features

$$Rv_i^l = \sum_j \frac{x_{kL}^l}{\sum_{0,k} x_{kL}^l} \widetilde{R}v_j^l, Rv_i^l = \{R_j^l | j = 0, \dots, n\}, X^l = \{x_j^l | j = 0, \dots, n\} \quad (6)$$

$$L = \widetilde{D}^{-\frac{1}{2}} \widetilde{A} \widetilde{D}^{-\frac{1}{2}}$$

$$\widetilde{X} = L X$$

1. find all neighboring nodes through L
2. propagate the relevance scores to all neighboring nodes in graph

Q & A

Setting

Datasets :

Citation network datasets : Citeseer, Cora and Pubmed

Experiment setup :

1. train a GCN model for semi-supervised classification on citation networks data sets
2. use GCN-LRP to explain the trained GCN model

Effectiveness of GCN-LRP explanation

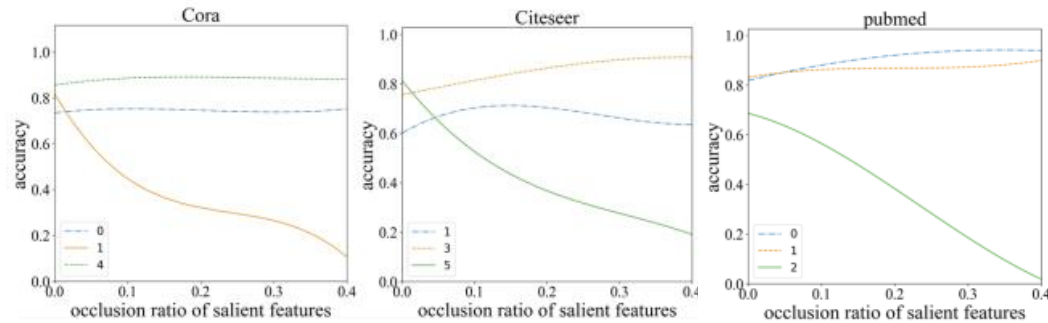
Explanation on node level

comparing contribution ratio of classified node with other nodes

comparing contribution ratio of neighboring nodes

Effectiveness of GCN-LRP Explanation

Impact of category-special feature occlusion



Cora : salient features of category 1 are occluded, compared for classification accuracy changing of category 1, category 0 and category 4

Citeseer : salient features of category 5 are occluded, compared for classification accuracy changing of category 5, category 1 and category 3

pubmed : salient features of category 2 are occluded, compared for classification accuracy changing of category 2, category 0 and category 1

Occlusion for each category in Cora

	Classification Accuracy						
	Cate gory. 1	Cate gory. 2	Cate gory. 3	Cate gory. 4	Cate gory. 5	Cate gory. 6	Cate gory. 7
Baselin e	0.79	0.89	0.92	0.76	0.86	0.76	0.77
Occludi ng-F1	0.18	0.84	0.99	0.80	0.90	0.78	0.73
Occludi ng-F2	0.74	0.13	0.85	0.71	0.86	0.78	0.73
Occludi ng-F3	0.77	0.86	0.15	0.53	0.91	0.76	0.61
Occludi ng-F4	0.81	0.88	0.83	0.18	0.67	0.83	0.73
Occludi ng-F5	0.61	0.82	0.97	0.55	0.25	0.77	0.93
Occludi ng-F6	0.67	0.59	0.96	0.77	0.94	0.12	0.67
Occludi ng-F7	0.47	0.85	0.94	0.77	0.96	0.69	0.22

Occlusion for Each Category in CORA

detail results of the feature occlusion in Cora

: baseline is the classification accuracy for each category with occlusion, and the following row contains the accuracy when we occlude the features of corresponding category

Explanation on node level

comparing contribution ratio of classified node with other nodes

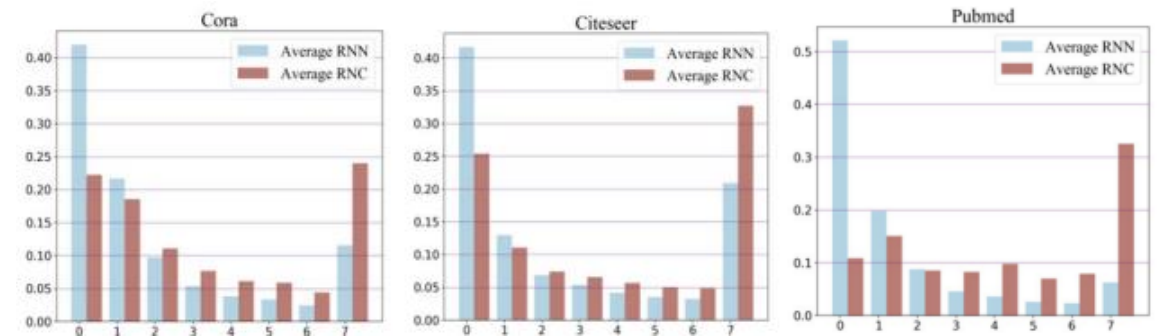
	Median CR	Mean CR	Std CR	CR of Classified node (ratio of being the largest one)
Cora	0.0843	0.0951	0.0459	0.1844 (72.1%)
Citeseer	0.1867	0.1965	0.0607	0.2865 (72.2%)
Pubmed	0.0384	0.0473	0.0294	0.1405 (84.5%)

Node Contribution ratio

Calculate the median, mean, standard deviation of CR(Contribution Ratio) from all nodes in each local group, and calculate CR for every node classification task → take average ratio

CR of classified node is always larger than the mean contribution ratio of all nodes in its local group, and it has over 72% chance to be the largest CR of nodes in all local groups from the three citation network data sets

comparing contribution ratio of neighboring nodes



For eight subintervals, left bar represents average RNN of each subinterval, and right bar represents average RNC of each subinterval

$$\text{RNN}(\text{ratio of node number}) = N_i / N_{all}$$

$$\text{RNC}(\text{ratio of node contribution}) = C_i / C_{all}$$

N_i : number of the nodes whose contribution value is in the subinterval

N_{all} : number of all neighboring nodes

C_i : total contribution of nodes whose contribution value is in the subinterval

C_{all} : total contribution of all neighboring nodes

- Suggest a new LRP based explanation method for GCNs
- GCN-LRP was applied to citation network data sets to explore the latent attention of GCNs
- With the experiments, it was found that the classified node and a few special neighbors would receive more attention than other nodes
- With further theoretical analysis, it was found that the latent attentions come from the recursively aggregating of GCNs and the attentions could hardly be changed by model training
- findings indicate GCNs would be more powerful when the characteristics of the learning tasks and data fit the latent attentions, and conversely, GCNs would be difficult for the learning tasks

Thank you ツ